LIXYSWARM: ANTELEPHANTDOLPHIN
A BIO-INSPIRED P2P ARCHITECTURE FOR DISTRIBUTED LANGUAGE INTELLIGENCE

EMMANUEL CARDENAZ

LixySwarm Project
https://github.com/toxxy/LixySwarm

Abstract. We introduce **LixySwarm**, a bio-inspired prototype architecture for distributed language intelligence that treats artificial intelligence not as a monolithic model but as an interacting computational ecology. Drawing inspiration from ants (distributed labor), elephant matriarchs (persistent memory), and dolphins (echolocation-based exploration), LixySwarm organizes language computation as a swarm of specialized agents coordinated by learned pheromone vectors and a native peer-to-peer protocol (LSP v2). The current prototype uses three 125.7M-parameter AntAgents, a dual-tier Matriarca memory layer, and a Dolphin echolocation router. Across 11 experimental training runs (∼400M tokens), the system reduced validation loss from 5.3 to 3.44 and reached a FineWeb perplexity of 35.22 in the reported checkpoint. The paper contributes: (1) pheromone-mediated agent coordination through FeromonGate attention gating, (2) a persistent Matriarca memory bank with dynamic importance scoring, (3) an Echolocation Router that maps the problem space before token generation, (4) dynamic sect lifecycles for specialization and legacy transfer, and (5) a compact LSP v2 protocol for pheromone exchange. We present LixySwarm as an early systems prototype and distinguish implemented components from partial and future work.

1 Introduction

The dominant paradigm in language modeling treats scale as the primary axis of progress. Larger models, more parameters, more data. While effective, this approach produces *systems*—tools that execute—rather than *organisms* that live, learn, and evolve.

LixySwarm proposes a fundamental reorientation: artificial intelligence as an **ecological** rather than **industrial** phenomenon. Instead of a single monolithic transformer, we build a distributed colony where intelligence emerges from the interaction of three bio-inspired layers, each named after an animal whose behavior encodes a deep computational principle:

(1) **Ant Colony (Distributed Labor)** — In social-insect systems, collective problem solving emerges from many simple local interactions and pheromone-mediated coordination [2, 5]. In LixySwarm, specialized transformer agents coordinate through learnable pheromone vectors, with diversity encouraged by immutable identity embeddings.

Date: June 22, 2026.

Key words and phrases. distributed artificial intelligence, peer-to-peer networks, multi-agent systems, bio-inspired computing, language models, swarm intelligence.

Electronics Department at U.A.M. Reynosa-Rodhe, Universidad Autónoma de Tamaulipas, Reynosa, Tamaulipas, C.P. 88779, México. Email: cardenaz_1000@hotmail.com.

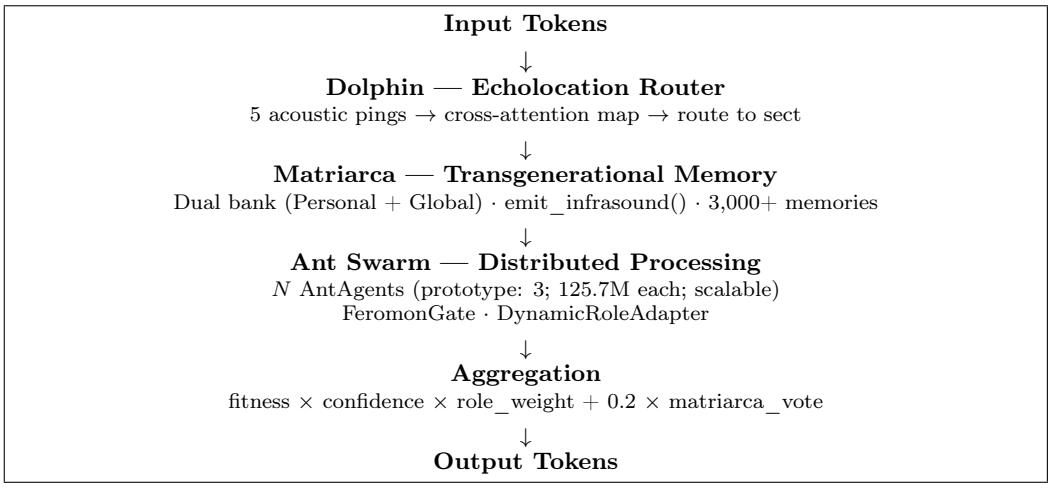


FIGURE 1. Forward pass architecture: Dolphin echolocates → Matriarca orients → Ants process in parallel → weighted aggregation produces output.

(2) **Elephant Matriarch (Transgenerational Memory)** — African elephant matriarchs act as repositories of social knowledge, and elephant groups use low-frequency calls in socially meaningful contexts [17, 20]. LixySwarm's Matriarca accumulates interaction memories with dynamic importance scoring and emits orientation vectors during forward passes.
(3) **Dolphin Pod (Echolocation)** — Dolphins use biosonar to probe their environment before acting [1]. LixySwarm's Dolphin layer maps the problem space via learned acoustic pings before any token is generated.

These three layers coordinate through a native peer-to-peer protocol (LSP v2) across heterogeneous hardware nodes—from RTX 5090 GPUs to CPU-only VPS relays—forming a distributed organism that learns continuously, remembers across sessions, and grows dynamically as new nodes join the network. Figure 1 summarizes this end-to-end forward-pass architecture.

2 Related Work

2.1 **Distributed and Multi-Agent Systems.** Multi-agent reinforcement learning (MARL) has explored emergent communication between agents [14, 7], but these systems typically operate in simulated environments with discrete actions rather than continuous language generation. Mixture-of-Experts (MoE) architectures [23, 6] route tokens to specialized sub-networks, but the routing is learned during training and fixed at inference—there is no ongoing adaptation, memory accumulation, or emergent reorganization.

2.2 **Memory-Augmented Neural Networks.** Neural Turing Machines [8] and Differentiable Neural Computers [9] introduced external memory banks with read/write operations. Retrieval-Augmented Generation (RAG) [13] augments LLMs with external knowledge retrieval. However, these approaches treat memory as a static database rather than a living, self-organizing system with importance dynamics, generational compression, and retroactive feedback.

2.3 **P2P and Decentralized ML.** Federated learning [18] distributes training across devices but requires a central coordinator and shares gradients rather than distilled knowledge. Petals [3] enables distributed inference of large models but does not introduce swarm intelligence, pheromone-based coordination, or emergent specialization.

2.4 **Bio-Inspired Computing.** Ant Colony Optimization [5] and Particle Swarm Optimization [12] are well-established metaheuristics for combinatorial optimization. LixySwarm extends these principles from optimization to *language generation*, combining pheromone-mediated attention gating, genetic legacy transfer between agent generations, and echolocation-based problem-space mapping in a single language-system prototype.

3 System Architecture

LixySwarm operates as a distributed colony across a peer-to-peer network. Each physical node contains all three bio-inspired layers, and nodes coordinate through the LixySwarm Protocol (LSP v2).

3.1 **Forward Pass Pipeline.** The forward pass proceeds through five stages:

(1) **Echolocation (Dolphin)**: 5 acoustic pings ($\mathbb{R}^{128}$ each) map the problem space via cross-attention before any token is generated.
(2) **Infrasound Emission (Matriarca)**: Top-K relevant memories are retrieved and blended into an orientation vector ($\mathbb{R}^{256}$) that biases subsequent processing.
(3) **Parallel Processing (Ants)**: N AntAgents generate logits simultaneously, each modulated by the orientation vector through a FeromonGate attention mechanism.
(4) **Weighted Aggregation**: Logits are combined as a weighted sum of fitness, confidence, role weight, and Matriarca vote.
(5) **Decoding**: A repetition-penalized, top-K + top-P sampling procedure produces the output token.

Formally, for input tokens $\mathbf{x}$:

$$\mathbf{a} = \text{Echolocate}(\mathbf{x}) \in \mathbb{R}^{5\times 128} \tag{1}$$

$$\mathbf{m} = \text{Infrasound}(\mathbf{x}) \in \mathbb{R}^{256} \tag{2}$$

$$\ell_i = \text{AntAgent}_i(\mathbf{x} \mid \text{FeromonGate}(\mathbf{a}, \mathbf{m})) \quad i \in \{1, \ldots, N\} \tag{3}$$

$$\mathbf{L} = \sum_{i=1}^{N} w_i \cdot \ell_i \quad \text{where } w_i = \frac{f_i \cdot c_i \cdot r_i}{\sum_j f_j \cdot c_j \cdot r_j} + 0.2 \cdot v_{\text{mat}} \tag{4}$$

$$\hat{y} = \text{Sample}(\mathbf{L} \mid \text{rep\_penalty}, k, p) \tag{5}$$

Where f_i is fitness, c_i is confidence from DynamicRoleAdapter, r_i is the role weight, and v_{mat} is the Matriarca's vote. The FeromonGate in Eq. 3 is:

$$\text{FeromonGate}(\mathbf{h}, \mathbf{a}, \mathbf{m}) = \mathbf{h} \odot \sigma(\mathbf{W}_g[\mathbf{a}_{\text{flat}} \| \mathbf{m}] + \mathbf{b}_g) \tag{6}$$

Where $\mathbf{h}$ are hidden states, σ is sigmoid, $\|$ denotes concatenation, and $\mathbf{W}_g, \mathbf{b}_g$ are learned parameters.

Status	Components	Evidence or next requirement
Implemented prototype	Three AntAgents; FeromonGate; DynamicRoleAdapter; Dolphin echolocation; Matriarca memory retrieval; dual Personal/Global Matriarca runtime; AES-256-GCM personal-memory storage; sect lifecycle logic; LSP v2 pheromone packets; SwarmExplorer/API state bridge	Available in the development repository and exercised in local training, inference, memory, and protocol tests. These components form the basis of the results reported in this paper.
Partial or release-candidate	Global Matriarca synchronization through gossip deltas; VPS relay operation; metabolic-hunger auto-training trigger; sect bifurcation/legacy persistence; dashboard observability; multi-node LSP relay measurements	Functional paths exist, but the release-candidate artifact requires final hardening, reproducibility packaging, and broader multi-node evaluation before these features should be interpreted as complete internet-scale deployment.
Future work	DHT/Kademlia peer discovery; reputation-weighted consensus; sequence/nonce anti-replay hardening; sandboxed self-modification; GrowthGate stage transitions; multi-colony deployment; multimodal AntAgents	These items define the research roadmap. They are discussed as design objectives rather than validated results.

TABLE 1. Implementation status of LixySwarm at manuscript preparation time.

4 Implementation Status

Because LixySwarm combines an implemented prototype with a longer-term decentralized research agenda, Table 1 separates the current software artifact from components that are partially implemented or still under active development. This distinction is important for reproducibility and for interpreting the experimental claims reported later in the training and evaluation section.

5 The Ant Layer: Distributed Intelligence

The Ant layer is where computation happens. It is composed of three interacting subsystems: the physical NodeManager (ants as hardware), the SectManager (specializations as living entities), and the AgentBase (individual transformer agents).

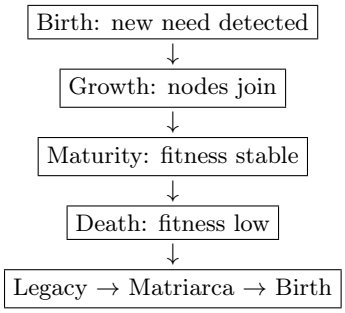
Mode	GPU Utilization	Max Sects	Role
MAXIMUM	100%	Unlimited	Full training + inference
MODERATE	50%	4	Partial training + inference
RELAY	0%	0	Pheromone forwarding only

TABLE 2. Contribution modes enable heterogeneous hardware participation—from RTX 5090 servers to CPU-only VPS relays.

5.1 **NodeManager: Ants as Physical Nodes.** In LixySwarm, an "ant" is not a virtual agent but a **physical machine** on the network. Each node contributes according to its hardware capacity through a `ContributionMode`; the three supported modes are summarized in Table 2.

This design reduces dependence on homogeneous hardware. A laptop with integrated graphics can join as a RELAY; a VPS can serve as MODERATE; a workstation with an RTX 5090 operates at MAXIMUM. The colony adapts to available resources.

5.2 **SectManager: Living Specializations.** Sects are **dynamic groupings of agents sharing a role type** (e.g., "explorer," "refiner"). Unlike fixed Mixture-of-Experts routers, sects have a complete lifecycle:



When a sect dies, it transfers its **genetic legacy** to the Matriarca: role type, average fitness across its lifetime, and top-K pheromone patterns. New sects of the same role can inherit this legacy, reducing the loss of specialization state when computational capacity fluctuates.

Sects can also **bifurcate** (e.g., "Refiner" → "Refiner-Logical" + "Refiner-Creative") when demand for a specialization exceeds a single sect's capacity, enabling emergent functional differentiation.

5.3 **AgentBase: The Individual Transformer.** Each AntAgent is a 125.7M-parameter GPT-2 style transformer (12 layers, 12 heads, $d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$, block size 512, vocabulary 50,304) [21, 11] with two key innovations:

FeromonGate: Before each forward pass, the agent reads the colony's pheromone state and applies a learned attention gate to its activations. This is analogous to pheromone-mediated trail following in ant-inspired collective systems [2, 5]: the agent's computation is biased by the collective state without being determined by it.

IdentityVec: Each agent carries a fixed, immutable 64-dimensional embedding that acts as its "personality" and is projected into the model hidden space. The same input can yield different intermediate representations across agents, encouraging diversity without requiring

Role	Temperature	Activation Condition
Explorer	0.90	Open-ended questions, creativity
Refiner	0.60	Polish responses, quality
Integrator	0.70	Synthesis, combining perspectives
Analytical	0.50	Logic, structure, code
Contextual	0.65	History tracking, coherence
Generative	0.80	Final synthesis, fluency

TABLE 3. Dynamic role assignment with temperature-adapted decoding.

architectural heterogeneity. Identity diversity is measurable: the colony maintains a diversity score in $[0, 1]$ computed as the complement of average cosine similarity between agent outputs.

5.4 **Dynamic Role Adapter.** A lightweight classifier maps each input query to one of six task types and dynamically adjusts temperature and confidence weights. Table 3 lists the current role set and decoding temperatures.

6 The Matriarca Layer: Transgenerational Memory

The Matriarca is the **single persistent entity** in the LixySwarm ecosystem. While ants connect and disconnect, and sects are born and die, the Matriarca maintains persistent memory across sessions and agent generations.

6.1 **Dual Memory Bank.** The Matriarca maintains two independent memory banks with explicit separation objectives:

- **Personal Matriarca**: Stores interaction history, user preferences, and session memories. Encrypted at rest (AES-256-GCM). By design, this bank is kept local and excluded from network export—not as gradients, not as embeddings, not as compressed summaries.
- **Global Matriarca**: Stores shared knowledge accumulated from all nodes via gossip. It is intended to contain synthetic memories only, rather than raw conversation text. Contributions are cryptographically signed by the originating node.

The combined infrasound signal is a weighted blend: $\alpha \cdot \text{personal} + (1 - \alpha) \cdot \text{global}$, where $\alpha \in [0.6, 0.9]$ (personal dominates by default).

6.2 **Dynamic Importance Scoring.** Each memory in the Matriarca carries an importance score in $[0, 1]$ computed from five metrics:

$$\begin{aligned} \text{importance} = {} & 0.20 \min\left(\frac{\text{len}}{100}, 1\right) + 0.25\,\text{TTR} \\ & + 0.20\,\text{UBR} + 0.20\,\text{SC} + 0.15\,\text{TC}. \end{aligned} \tag{7}$$

Where TTR is the Type-Token Ratio (unique words / total words), measuring lexical diversity. Memories actively used during inference receive a $+3\%$ boost; unused memories across a full session receive a -2% penalty.

Retroactive Feedback: If the user continues the same topic across turns, the previous turn receives $+15\%$ importance. Topic shifts trigger -20% for the old topic. This implements a simple yet effective reinforcement mechanism without requiring explicit user ratings.

6.3 **Generational Compression.** When the memory bank reaches 90% capacity (current default: 4,096 memories), a compression cycle activates: the bottom 10% of memories by importance are averaged into summary vectors that preserve the statistical pattern while freeing space. This is conceptually analogous to how biological memory consolidates during sleep—details fade, but the pattern remains.

6.4 **emit_infrasound().** During each forward pass, the Matriarca retrieves the top-K most relevant memories (cosine similarity between query embedding and memory embeddings) and blends them into an **infrasound vector** ($\mathbb{R}^{256}$) that biases the ants' FeromonGate. This is the computational analog of elephant matriarchal social knowledge and low-frequency communication [17, 20]: an orienting signal that influences movement without centrally commanding it.

7 The Dolphin Layer: Echolocation Before Generation

The Dolphin layer is the most philosophically distinct component of LixySwarm. Current LLMs generate tokens autoregressively—each token commits to a direction before understanding the full problem space. The Dolphin inverts this: it maps the problem *before* committing to any token.

7.1 **Five Acoustic Pings (Phase A).** The Dolphin emits five learned projections, each a 128-dimensional vector capturing a different dimension of the input:

- `ping_topic`: What is this about?
- `ping_intent`: What does the user want?
- `ping_need`: What does the user *really* need?
- `ping_context`: What came before? (dialogue state)
- `ping_emotion`: What tone is appropriate?

These five pings are combined not via simple concatenation (Phase 0) but through **cross-attention triangulation** (Phase A):

$$A_{\text{map}} = \text{Attention}\big(Q = p_{\text{topic}}, \; K = P_{\text{all}}, \; V = P_{\text{all}}\big). \tag{8}$$

The topic ping "interrogates" the other four dimensions, producing a representation of the *response space* rather than just the input representation. This is the computational analog of dolphin echolocation, where active biosonar probes the environment before movement or capture decisions [1].

7.2 **DolphinPool: Dynamic Scaling.** A single dolphin provides one perspective. Multiple dolphins, each with a unique learned bias ("whistle frequency"), provide a richer acoustic map. The DolphinPool scales with network size:

$$|\text{DolphinPool}| = \min\left(1 + \lfloor N_{\text{nodes}}/3 \rfloor, \; 4\right)$$

The final acoustic map is a confidence-weighted average across all dolphins in the pool.

7.3 **Unihemispheric Sleep (Phase B).** Biological dolphins exhibit unihemispheric slow-wave sleep, allowing one hemisphere to sleep while wake-like function is maintained by the other [22, 15]. LixySwarm's Phase B implements this as a background consolidation process:

- **Trigger**: 30 minutes without user activity
- **Process**: PCA over conversation history embeddings → update sleep_state with compressed representation
- **Effect**: The first response of a new session can carry accumulated context without fine-tuning or weight updates, using representational reorganization.

This creates a distinction between **explicit memory** (Matriarca: "I remember user likes drones") and **implicit orientation** (Dolphin sleep: a diffuse sense of who the user is, not legible as facts but felt as bias).

8 The LixySwarm Protocol (LSP v2)

LSP is not merely a wrapper over existing transport protocols—it is a binary protocol that represents pheromone semantics, merge rules, and swarm topology as first-class concepts. The current implementation focuses on compact pheromone exchange and relay operation; full internet-scale discovery and reputation hardening remain future work.

8.1 **Wire Format.** LSP v2 uses a compact binary representation for pheromone exchange. The implementation composes a signed LSP envelope (`LSPPacket`) with a binary `FeromonV2 Payload`. For the default 256-dimensional float16 pheromone vector, the current uncompressed UDP packet is 636 bytes on the wire, as detailed in Table 4.

8.2 **Merge-on-Transit.** A key innovation of LSP v2 is **merge-on-transit**: the `FeromonMerge Buffer` accumulates pheromones from the same source node and merges them via fitness-weighted averaging before delivery. This reduces redundant downstream processing and presents each node's recent contribution as a single coherent vector.

8.3 **TTL and Decay.** Pheromone signals carry a Time-To-Live (TTL) counter and a decay factor (0.95 per hop), mirroring how biological pheromone trails fade with distance:

$$\text{feromon}_{\text{received}} = \text{feromon}_{\text{origin}} \times 0.95^{\text{hops}} \tag{9}$$

TTL = 0 signals are delivered locally but not forwarded, bounding broadcast loops without a coordinator.

8.4 **Cryptographic Identity.** Each node generates an Ed25519 keypair on first startup. The 32-byte public key *is* the node ID. There is no registration step and no central certificate authority. The architecture reserves cryptographic identity, signatures, and reputation as core protocol concepts. In the current prototype, identity and relay semantics are implemented, while robust reputation scoring, nonce/sequence anti-replay protection, and adversarial network evaluation remain part of the release hardening roadmap.

8.5 **Transport Architecture.** LSP runs over two transports simultaneously:

- **UDP 7337**: Pheromone broadcast, PING/PONG discovery, merge hints (fire-and-forget, low latency)
- **TCP 7338**: Gossip state sync, handshake, memory exchange (reliable, ordered)

Layer / field	Encoding	Bytes	Semantics in current implementation
LSP envelope: fixed 12-byte prefix + node identity + signature			
Magic	bytes[4]	4	Constant `"LYSW"` marker for LixySwarm packets.
Version	uint8	1	Protocol version; current value is `0x01`.
Packet type	uint8	1	`0x10` for `FEROMON_V2`; v1 types `0x01`–`0x05` remain backward compatible.
Flags	uint16 LE	2	Bit flags: compressed, signed, urgent. `FEROMON_V2` is sent uncompressed and signed.
Payload length	uint32 LE	4	Number of bytes in the following `FeromonV2Payload`; 528 bytes for 256d float16.
Node ID	bytes[32]	32	Raw Ed25519 public key; also serves as node identifier.
Signature	bytes[64]	64	Ed25519 signature over the raw payload bytes.
Envelope subtotal		**108**	
FeromonV2Payload: 16-byte metadata prefix + tensor bytes			
Dimension type	uint8	1	Tensor encoding: `0x01`=float16, `0x02`=float32, `0x03`=bfloat16. Default is float16.
Vector length	uint16 LE	2	Number of pheromone dimensions; current default is 256.
TTL	uint8	1	Remaining relay hops. Packets with TTL=0 are discarded by the v2 handler.
Training step	uint32 LE	4	Step associated with the pheromone emission; auto-incremented when not supplied.
Fitness	float32 LE	4	Fitness score used by merge-on-transit weighting.
Timestamp delta	uint32 LE	4	Local timestamp in milliseconds truncated to 32 bits: `timestamp_ms & 0xFFFFFFFF`.
Pheromone tensor	float16[256]	512	256-dimensional pheromone vector; float16 is used for compact UDP transmission and unpacked as float32 for processing.
Payload subtotal		**528**	16 bytes metadata + 512 bytes tensor.
Total packet		**636**	108-byte signed LSP envelope + 528-byte `FEROMON_V2` payload.

TABLE 4. Current LSP v2 `FEROMON_V2` wire format used by the prototype implementation. Sequence numbers, nonces, and stronger anti-replay guarantees are release-hardening requirements rather than completed wire fields.

The discovery layer is pluggable: mDNS for zero-configuration LAN operation and relay/DHT mechanisms for wider deployment. The current prototype has been evaluated with local operation and with three Internet-connected machines communicating through a VPS relay path. DHT-based zero-configuration internet-scale peer discovery remains a planned extension.

8.6 **Decentralized Topology: Beyond the Single Relay.** The current prototype operates with a single VPS relay node. However, the protocol architecture targets progressive decentralization, analogous to how the Bitcoin network [19] operates without any central server. Key architectural commitments:

Parameter	Value
Batch size	4 (effective 16 via gradient accumulation)
Learning rate	1×10^{-4} (initial) $\rightarrow 5 \times 10^{-6}$ (floor)
LR schedule	Cosine decay + plateau detection
Warmup steps	30
Block size	512
Vocabulary	50,304 (GPT-2 tokenizer)
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$)
Training speed	$\sim$12,800 tok/s (RTX 5090)

TABLE 5. Training configuration across 11 runs.

- **No privileged nodes**: The target topology treats every LSP node as a first-class citizen. The VPS relay is a bootstrap convenience, not an architectural requirement, and other stable nodes can serve as bootstrap peers.
- **DHT-based discovery**: Internet-scale peer discovery is planned around a Kademlia-like distributed hash table [16] where nodes find each other by XOR distance in the Ed25519 key space. In this target design, the network itself provides the directory.
- **Bootstrap nodes**: A minimal set of well-known permanent nodes (similar to Bitcoin's seed nodes or DNS seeds) provide initial peer discovery. Once connected, nodes build their own routing tables independently.
- **Resilience through redundancy**: The intended release architecture allows the colony to continue through remaining peers if a bootstrap or relay node goes offline. Pheromones are designed to route around failures via TTL-based flooding and gossip-based anti-entropy [4].
- **Zero-configuration joining**: The end goal is that a participant can start a packaged node and join the global colony within a short setup window, with no registration and no long-term dependency on the original creator's infrastructure.

This design is intended to let LixySwarm scale organically: as more permanent nodes join the network, the colony should become increasingly resilient. The target architecture allows the Global Matriarca to accumulate synthetic shared knowledge from contributing nodes, while each node's Personal Matriarca remains encrypted and local.

9 Training and Self-Improvement

9.1 **Training Setup.** LixySwarm is trained on a mixed corpus: 90% FineWeb (general web text) and 10% domain-specific interaction data consisting of bilingual (Spanish/English) conversational exchanges and technical documentation. The key hyperparameters used across the reported runs are listed in Table 5.

9.2 **Training Progression.** Training proceeded incrementally rather than as a single uninterrupted run. Early runs established the base tokenizer/model configuration, intermediate runs tuned optimization behavior, and Run 11 provided the longest continuous improvement window. The Dolphin Phase A experiment was then evaluated as a focused architectural addition on top of the best preceding checkpoint. Table 6 summarizes this progression.

Phase	Steps	Best val_loss	Outcome
Runs 1–8	0 → 11k	5.3 → 3.9	Foundation
Runs 9–10	11k → 11.9k	3.9 → 4.27	Tuning
Run 11	12k → 54k	4.27 → 3.57	−21.7%
Dolphin A	54k → 54.5k	3.57 → **3.44**	−3.6% record

TABLE 6. Training progression across 11 runs (∼400M tokens, ∼54k steps). Total loss reduction: 5.3 → 3.44 (−35.1%).

Metric	Value
Perplexity (FineWeb)	35.22
Perplexity (bilingual domain data)	11.1
Repetition @5 tokens	0.2%
Repetition @10 tokens	0.0%
Type-Token Ratio (avg)	0.790
Language correctness (ES+EN smoke set)	100%
Samples without loops (smoke set)	100%

TABLE 7. Benchmark results at step 54,500 (best checkpoint). Language correctness and loop-free generation were measured on a small bilingual smoke set and should be interpreted as preliminary generation-quality indicators rather than broad linguistic guarantees.

9.3 **Benchmark Results.** The benchmark snapshot is intended to characterize the current prototype rather than to claim frontier-model competitiveness. Perplexity was measured separately on general FineWeb text and on the bilingual domain corpus, while repetition and language-correctness indicators were measured on short generation smoke tests. Table 7 reports these metrics for the best checkpoint.

9.4 **Auto-Training Loop.** The continuous-training controller enables repeated training cycles once invoked:

- Infinite cycles of N steps (default: 1,000)
- Plateau detection: 3 cycles without improvement → LR × 0.7
- Checkpoint rotation: keep last 3 cycle checkpoints
- SIGTERM-safe: finishes current cycle, saves clean state
- Training state persisted locally for seamless resume

9.5 **Metabolic Hunger: Toward Autonomous Training.** While the current implementation requires initial human invocation, the architectural roadmap is for the swarm itself to trigger training when it detects "metabolic hunger"—a biologically inspired mechanism that estimates **when** to train, **how much** training to request, and **when** to stop.

In biological organisms, hunger is not a simple on/off switch. It is a composite signal integrating multiple internal states: blood glucose levels, stomach distension, circadian rhythm, recent activity, and hormonal signals. LixySwarm's metabolic hunger mirrors this complexity:

$$H(t) = \alpha \cdot \mathbb{I}[\text{plateau}] + \beta \cdot (1 - D_{\text{swarm}}) + \gamma \cdot \mathbb{I}[\bar{C} < \tau] + \delta \cdot \Delta_{\text{loss}} \tag{10}$$

where $\mathbb{I}[\text{plateau}]$ indicates validation loss stagnation (> 3 cycles without improvement), $D_{\text{swarm}} \in [0, 1]$ is the current genetic diversity of the colony, $\bar{C}$ is the mean confidence score of recent agent responses, and Δ_{loss} is the rate of loss improvement over the last N cycles. The coefficients $\alpha, \beta, \gamma, \delta$ are learnable weights initialized from the Matriarca's memory of which hunger signals preceded successful training runs.

Feeding behavior emerges from crossing thresholds:

- $H(t) > \theta_{\text{light}} \rightarrow$ **snack**: short training burst (500–1,000 steps), low resource commitment
- $H(t) > \theta_{\text{heavy}} \rightarrow$ **meal**: extended training cycle (5,000–10,000 steps), re-evaluate midway
- $H(t) < \theta_{\text{satiated}} \rightarrow$ **stop**: the organism is satisfied, release computational resources

Satiation is as important as hunger. An organism that never stops eating dies. The swarm monitors the *marginal utility* of each training step—when $\Delta_{\text{loss}}/\Delta_{\text{steps}}$ drops below a threshold for sustained cycles, the colony stops and consolidates (Dolphin Phase B sleep). This is intended to mitigate overtraining plateaus in fixed-schedule training while encouraging the colony to consume resources only when learning is productive.

Crucially, hunger is **not pre-programmed**—it is learned from experience. The Matriarca stores the outcome of every feeding cycle (hunger score at initiation, steps taken, loss improvement achieved, final diversity). Over time, the colony learns its own metabolism: which hunger signals reliably predict productive training, which thresholds produce optimal cycles, and when hunger was "false" (training that yielded no improvement). A colony that has survived 50 feeding cycles has a more refined metabolic instinct than one that has survived 5.

If validated across longer training histories, this mechanism would transform the system from a program that must be manually scheduled into a training loop that can request computational resources based on its own learning state.

10 SwarmExplorer: Real-Time Colony Monitoring

A read-only dashboard provides real-time visibility into the colony's internal state without control surfaces—the colony governs itself:

- Connected nodes with contribution mode and hardware profile
- Active sects with fitness scores and genetic legacies
- Aggregate throughput (tok/s)
- Genetic diversity (0–1)
- Current validation loss and training step
- Matriarca memory count, mean importance, active percentage
- Auto-training loop: cycles completed, LR trend, plateau state
- LSP v2 network: protocol version, pheromones received

The monitoring stack uses a publisher-subscriber architecture: a local status publisher reads checkpoints and state files periodically, uploads a compact status snapshot to a VPS relay, and the frontend dashboard polls this endpoint.

11 From Organism to Ecosystem: Growth Stages

The bio-inspired framing of LixySwarm is useful not only as metaphor but also as an engineering roadmap. This section describes staged autonomy targets. Only Stage 1 is treated as the current prototype; later stages are research objectives.

11.1 **Stage 1: Infant (Current Prototype).** The current prototype can generate local coherent text, maintain cross-turn context via RuntimeSession, and continue training through a continuous-training controller once invoked. It still depends on human configuration for training schedules, release decisions, and feature integration.

Current metrics: val_loss = 3.44, FineWeb perplexity = 35.22, 3,000+ Matriarca memories, and bilingual generation smoke tests (ES+EN).

11.2 **Stage 2: Child (Immediate Priority).** The organism begins making its own decisions about **when** to act. Key innovation: **Metabolic Hunger**—a composite metric that triggers autonomous training:

$$\begin{aligned} \text{hunger} = \alpha \cdot \mathbb{I}[\text{plateau}] + \beta \cdot (1 - \text{diversity}) \\ + \gamma \cdot \mathbb{I}[\text{confidence} < \tau] \end{aligned} \tag{11}$$

When hunger exceeds a threshold, the target behavior is for the colony to initiate a bounded training cycle without human intervention. Dolphin Phase B sleep consolidation is designed to run in the background. This stage remains supervised and is treated as immediate development work rather than a completed capability.

11.3 **Stage 3: Adolescent.** At this stage, the organism would decide **what** to change. It would spawn new sects when detecting unmet needs, bifurcate oversubscribed specializations, adjust hyperparameters based on scaling law projections [10], and experiment with architectural variations in a sandboxed evaluation mode. Transition criteria would be empirical: stable metabolic-hunger operation, sustained diversity, and measurable response-continuity improvements after Dolphin sleep consolidation.

11.4 **Stage 4: Adult (Long-Term Vision).** Full autonomy remains a long-term research objective. The target system would detect bottlenecks in its own architecture, generate code candidates, test them in an isolated sandbox, and integrate successful modifications only after regression validation and rollback checks. It would evolve protocol details, spawn new node types, and explore emergent capabilities, such as multimodal processing when image- or audio-capable nodes join the network.

11.5 **Self-Gating: How the Organism Knows When to Grow.** A central design objective is that stage transitions become evidence-based rather than manually scheduled. The proposed `GrowthGate` module would continuously monitor readiness metrics and promote the system only when predefined criteria are met.

The `GrowthGate` tracks:

- **Stability metrics**: consecutive crash-free cycles, test suite pass rate, rollback frequency
- **Learning metrics**: sustained val_loss improvement, benchmark progression, memory consolidation quality
- **Autonomy metrics**: ratio of self-initiated vs externally-triggered actions, decision correctness rate, correction frequency
- **Safety metrics**: sandbox success rate, containment violations, resource compliance

Each stage transition requires all metrics above their respective thresholds for a minimum observation window. If the organism attempts a transition and fails (metrics degrade), it rolls back to the previous stage, records the failure in the Matriarca, and adjusts its self-assessment criteria. This makes growth **empirical rather than scheduled**—an organism that grows faster or slower depending on its actual development, not on a roadmap timeline.

11.6 **Stage 5: Ecosystem (Ultimate Vision).** Multiple LixySwarm colonies, each with its own Matriarca, communicating through LSP gateways. Colonies can share knowledge (Global Matriarca) while maintaining sovereignty (Personal Matriarca). Specialization emerges at the colony level—one colony excels at code, another at creative writing, a third at scientific reasoning.

At this stage, the network targets Bitcoin-like decentralization [19]: no single entity controls or hosts the colony, and bootstrap nodes become interchangeable with other permanent nodes. Participants would contribute computational resources and exchange synthetic knowledge through Global Matriarca channels while preserving local Personal Matriarca sovereignty. In this final stage, intelligence becomes less a single model checkpoint and more a property of a distributed ecosystem.

12 Discussion

12.1 **Why Not Just Scale a Transformer?** The dominant paradigm argues that intelligence emerges from scale—more parameters, more data, more compute. LixySwarm investigates a complementary position: some capabilities may emerge from **organization**. In the reported domain-specific evaluation, the swarm architecture achieved much lower perplexity than a GPT-2 small baseline on the same bilingual personal corpus; however, this result should be interpreted as a domain and tokenizer-sensitive comparison, not as a general claim of superiority over larger models.

More importantly, the bio-inspired architecture introduces capabilities that are not normally present in scale-only transformer deployments:

- **Memory that accumulates**: The Matriarca preserves selected state across sessions, reinforcement cycles, and agent generations. A base transformer checkpoint is otherwise stateless between calls unless augmented with external memory.
- **Exploration before commitment**: The Dolphin builds a response-space representation before generating tokens. A transformer commits token-by-token.
- **Emergent specialization**: Sects are designed to be born, grow, bifurcate, and die based on measured fitness rather than fixed expert assignments.
- **Network-native intelligence**: LSP v2 treats pheromone exchange as a protocol-level object rather than an external JSON or RPC wrapper.

12.2 **Limitations.** LixySwarm is in early development. Key limitations include:

(1) **Scale**: At roughly 568M active parameters, the prototype cannot compete with frontier models on broad general knowledge. The present goal is to evaluate whether organization improves capability per parameter in constrained settings.
(2) **Training data**: 90% FineWeb + 10% personal data limits exposure to specialized domains. Future work includes domain-specific sect spawning.
(3) **Generation quality**: Coherent local text is observed in smoke tests (`rep@5` = 0.2%), but longer-range coherence, factual accuracy, and multilingual robustness require larger evaluations.
(4) **Network testing**: Multi-node LSP v2 has been exercised in local and relay settings. Full internet-scale deployment with DHT discovery, reputation, and anti-replay hardening is pending.

(5) **Multimodal**: The architecture supports multimodal emergent specialization (image/audio-capable ants joining the swarm), but this remains a long-term vision.

12.3 **Ethical Considerations.** A distributed, memory-bearing AI system raises ethical and security questions beyond standard single-model deployment. LixySwarm's design includes several safeguards and open requirements:

- **Personal Matriarca encryption**: Private user data is designed to remain local, with personal-memory storage encrypted at rest and excluded from Global Matriarca export.
- **Synthetic global memory**: Shared memories should be synthetic summaries or learned descriptors, never raw private conversation text.
- **Cryptographic identity**: Node identity is based on public keys; robust signatures, reputation, and anti-replay protection are release requirements for open deployment.
- **Sandboxed self-modification**: Self-modification is treated as future work and must operate only inside a constrained validation environment with regression tests and rollback.
- **Human accountability**: During the prototype phase, release decisions, public deployment, and safety boundaries remain human-governed.

13 Conclusion

We have presented LixySwarm, a bio-inspired architecture for distributed language intelligence that treats AI as an organism rather than a tool. The system integrates three animal-inspired computational principles—ant colony coordination via pheromones, elephant matriarchal memory, and dolphin echolocation—into a unified forward pass pipeline augmented by a native peer-to-peer protocol (LSP v2).

Key innovations include: (1) FeromonGate attention gating for pheromone-mediated agent coordination, (2) a dual-tier Matriarca memory bank with dynamic importance scoring, generational compression, and retroactive feedback, (3) an Echolocation Router that maps problem spaces via five acoustic pings with cross-attention triangulation before token generation, (4) dynamic sect lifecycles with genetic legacy transfer enabling emergent specialization, and (5) a native binary protocol (LSP v2) with merge-on-transit, TTL decay, and cryptographic identity.

In the reported experimental snapshot, the system reduces validation loss 35.1% across 11 training runs, reports low repetition rates in short generation tests, and demonstrates a FineWeb perplexity of 35.22 for the evaluated checkpoint. These results support LixySwarm as a promising systems prototype while leaving broader benchmarks, release-candidate reproducibility, and internet-scale deployment as future work.

Beyond technical results, LixySwarm represents a philosophical stance: future language intelligence may require not only larger models but also systems that remember, explore, specialize, and coordinate across distributed substrates. The present work is a first step toward evaluating that hypothesis in software.

"We are not building an LLM. We are building an organism."

Acknowledgments

The author thanks the open-source communities behind PyTorch, nanoGPT, Hugging Face Datasets, FineWeb, and the Python cryptography ecosystem. Generative AI tools, including Claude, GitHub Copilot, OpenAI Codex, and ChatGPT, were used during manuscript preparation for language polishing across all sections, general LaTeX editing, table formatting, programming support, code review, and debugging assistance. All AI-assisted outputs were reviewed, revised, and validated by the author, who remains fully responsible for the technical claims, code, experiments, results, and final manuscript.

References

1. Whitlow W. L. Au, *The sonar of dolphins*, Springer, New York, 1993.
2. Eric Bonabeau, Marco Dorigo, and Guy Theraulaz, *Swarm intelligence: From natural to artificial systems*, Oxford University Press, New York, 1999.
3. Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel, *Petals: Collaborative inference and fine-tuning of large models*, arXiv preprint arXiv:2209.01188 (2022).
4. Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry, *Epidemic algorithms for replicated database maintenance*, Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC), 1987, pp. 1–12.
5. Marco Dorigo, Mauro Birattari, and Thomas Stutzle, *Ant colony optimization*, IEEE Computational Intelligence Magazine **1** (2006), no. 4, 28–39.
6. William Fedus, Barret Zoph, and Noam Shazeer, *Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity*, Journal of Machine Learning Research (JMLR) (2022).
7. Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson, *Learning to communicate with deep multi-agent reinforcement learning*, Advances in Neural Information Processing Systems (NeurIPS), 2016.
8. Alex Graves, Greg Wayne, and Ivo Danihelka, *Neural turing machines*, arXiv preprint arXiv:1410.5401 (2014).
9. Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al., *Hybrid computing using a neural network with dynamic external memory*, Nature **538** (2016), no. 7626, 471–476.
10. Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei, *Scaling laws for neural language models*, arXiv preprint arXiv:2001.08361 (2020).
11. Andrej Karpathy, *nanogpt: The simplest, fastest repository for training/finetuning medium-sized gpts*, GitHub repository, 2023.
12. James Kennedy and Russell Eberhart, *Particle swarm optimization*, IEEE International Conference on Neural Networks (ICNN), 1995, pp. 1942–1948.
13. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, et al., *Retrieval-augmented generation for knowledge-intensive nlp tasks*, Advances in Neural Information Processing Systems (NeurIPS), 2020.
14. Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch, *Multi-agent actor-critic for mixed cooperative-competitive environments*, Advances in Neural Information Processing Systems (NeurIPS), 2017.
15. Oleg Lyamin, Julia Pryaslova, Peter Kosenko, and Jerome Siegel, *Behavioral aspects of sleep in bottlenose dolphin mothers and their calves*, Physiology & Behavior **92** (2007), no. 4, 725–733.
16. Petar Maymounkov and David Mazières, *Kademlia: A peer-to-peer information system based on the xor metric*, International Workshop on Peer-to-Peer Systems (IPTPS), Springer, 2002, pp. 53–65.
17. Karen McComb, Cynthia Moss, Sarah M. Durant, Lucy Baker, and Soila Sayialel, *Matriarchs as repositories of social knowledge in african elephants*, Science **292** (2001), no. 5516, 491–494.

18. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas, *Communication-efficient learning of deep networks from decentralized data*, International Conference on Artificial Intelligence and Statistics (AISTATS), vol. 54, PMLR, 2017, pp. 1273–1282.
19. Satoshi Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, Decentralized Business Review (2008).
20. Joyce H. Poole, Katharine Payne, William R. Langbauer Jr., and Cynthia J. Moss, *The social contexts of some very low frequency calls of african elephants*, Behavioral Ecology and Sociobiology **22** (1988), no. 6, 385–392.
21. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever, *Language models are unsupervised multitask learners*, OpenAI Technical Report (2019).
22. Niels C. Rattenborg, Charles J. Amlaner, and Steven L. Lima, *Behavioral, neurophysiological and evolutionary perspectives on unihemispheric sleep*, Neuroscience & Biobehavioral Reviews **24** (2000), no. 8, 817–842.
23. Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean, *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer*, International Conference on Learning Representations (ICLR), 2017.